Coding for Artificial Intelligence

Module 1, Week 3: Sequence Manipulation

Problem Set 3.1

Nested Loops, Short-Circuit Logic, and Flow Control

Project: The Sensor Anomaly Filter

Course	Coding for AI (Module 1, Week 3)
Author	Dr. Prateek, Assistant Professor, School of AI
Format	In-class practical, **1 hour 30 minutes** total, inclusive of write-up and submission
Total Marks	100 (+15 Bonus)

Read this document in full before you begin writing code.

Contents

1 Objective

Problem Set 3 manipulated a single, flat list. This practical introduces **nested loops** — a list of lists — and asks you to build a validation engine that reasons about data quality using `continue`, `break`, and the loop `else` clause: three flow-control tools that are frequently misunderstood, and doubly so once loops are nested.

By the end of this practical you should be able to:

(i) iterate over a nested structure using `range(len())` for index tracking,

(ii) use `continue` and `break` correctly inside nested loops, understanding precisely *which* loop each one affects, and

(iii) use a loop's `else` clause to detect whether that loop completed normally or was interrupted by `break`.

Time budget. This entire activity — coding, testing, and writing up your submission — is designed to fit in **90 minutes**. A suggested breakdown is given in Section 9.

2 The Problem: The Sensor Anomaly Filter

You are given a **Stream of Telemetry**: a list of lists, where each inner list is a *batch* of sensor readings. Your task is to audit this stream for **Drift Anomalies** — readings that exceed a safe threshold — while correctly handling two special string values that can appear in place of a numeric reading: `"ERR"` (noise, to be skipped) and `"STOP"` (an emergency signal that must halt the entire audit).

```
telemetry_stream = [
    [22.5, 23.0, 22.8],
    [25.1, "ERR", 24.9],
    [30.2, 35.5, 40.1],    # Threshold breach
    [22.0, 22.1, "STOP"], # Termination signal
]
```

3 Functional Requirements

Your script, `anomaly_auditor.py`, must:

1. **Iterate through batches.** Use a `for` loop, indexed via `range(len(telemetry_stream))`, to process each sub-list.
2. **Handle noise (`continue`).** If a reading is the string `"ERR"`, use `continue` to skip it and move to the next reading in that batch.
3. **Threshold logic.** If a numerical reading exceeds `35.0`, print: `"Anomaly Detected at Batch [Index]: [Value]"`.
4. **Emergency shutdown (`break`).** If the loop encounters the string `"STOP"`, the entire auditing process — not just the current batch — must terminate immediately. **Read Section 4 before implementing this: a single `break` is not sufficient.**
5. **Audit summary (`else`).** Print `"Audit Complete: No system-wide failures"` only if the audit finished without ever encountering `"STOP"`.

4 Critical Design Note: Breaking Out of Nested Loops

This is the single most important idea in this practical, so it is given its own section.

In Python, `break` terminates only the **innermost** loop it is written inside. In a nested structure, calling `break` when you find `"STOP"` exits the *inner* reading-loop for that batch only — the *outer* batch-loop will simply continue on to the next batch, which directly contradicts the requirement that `"STOP"` ends the entire audit.

The standard, idiomatic fix is a **flag variable**, checked immediately after the inner loop and used to `break` the outer loop as well:

```
shutdown_triggered = False

for batch_id in range(len(telemetry_stream)):
    batch = telemetry_stream[batch_id]

    for reading in batch:
        if reading == "STOP":
            print(f"Emergency Shutdown at Batch {batch_id}.")
            shutdown_triggered = True
            break                  # exits only the INNER loop

    if shutdown_triggered:
        break                      # now exits the OUTER loop too

else:
    print("Audit Complete: No system-wide failures")
```

Notice where the `else` clause is indented: it belongs to the **outer** `for` loop, not the inner one. Python only skips a loop's `else` block if *that specific loop* was exited via `break`. Because the outer loop is only broken when `shutdown_triggered` is `True`, the `else` block correctly prints the summary in every case *except* when `"STOP"` was seen — which is exactly the behaviour Requirement 5 asks for.

5 Technical Constraints

- **Indexing Mastery.** Use `range(len())` to track the **Batch ID** (the index of the outer list) while iterating. The inner loop does not need its own index; iterate over the batch's readings directly.
- **Type Robustness.** Use `isinstance(reading, (int, float))` to distinguish numeric readings from control strings **before** performing any comparison such as `reading > 35.0`. If you compare a string to a number without this check, Python raises a `TypeError`.

  > *Note.* `str.isdigit()` is not appropriate here. The numeric readings in this dataset are already native `float` values, not strings that need parsing — there is nothing to call `.isdigit()` on. And even if the readings arrived as strings, `.isdigit()` returns `False` on any string containing a decimal point (e.g., `"35.5".isdigit()` is `False`), so it would reject valid sensor readings outright. `isinstance()` against the actual Python types is the correct tool for this job.

- **Order of checks matters.** Test for `"STOP"` first, then `"ERR"`, and only then treat the value as numeric. Checking in a different order risks attempting a numeric comparison on a control string.

- **PEP 8 Compliance.** Consistent 4-space indentation throughout; misaligned blocks will be penalised, and in nested loops a misplaced `else` silently attaches to the wrong loop rather than raising an error, which makes this doubly important here.

6 Worked Example / Trace Log Reference

Running a correct implementation against the dataset in Section 2 produces the following trace. Use it to verify your own output line for line.

```
--- Auditing Batch 0: [22.5, 23.0, 22.8] ---
--- Auditing Batch 1: [25.1, 'ERR', 24.9] ---
Noise ignored at Batch 1 (ERR).
--- Auditing Batch 2: [30.2, 35.5, 40.1] ---
Anomaly Detected at Batch 2: 35.5
Spike Detected at Batch 2: 30.2 -> 35.5 (Delta 5.3)
Anomaly Detected at Batch 2: 40.1
--- Auditing Batch 3: [22.0, 22.1, 'STOP'] ---
Emergency Shutdown at Batch 3.
```

Note that no `"Audit Complete"` line appears: `"STOP"` was encountered in Batch 3, so the outer loop's `else` clause is correctly skipped. The two `Spike Detected` and `Anomaly Detected` lines both concern Batch 2 — the Rolling Delta check described below is what produces the `Spike Detected` line.

7 Challenge Task (Bonus, +15 Marks)

Implement a **Rolling Delta** check. Within each batch, compare every numeric reading to the *previous numeric reading in that same batch.* If the absolute difference exceeds `5.0`, flag it as a `"Spike"`.

> *Hint.* Store the previous value in a variable declared inside the **outer** loop but outside the **inner** loop, so that it resets to "no previous reading" (`None`) at the start of every new batch, rather than carrying over from the batch before. Update it only after a successful numeric comparison, so that skipped values (`"ERR"`) do not silently reset your delta tracking.

Using the dataset in Section 2, Batch 2 (`[30.2, 35.5, 40.1]`) should report exactly one spike: $|35.5 - 30.2| = 5.3 > 5.0$. The step from 35.5 to 40.1 has a delta of only 4.6, which does *not* qualify.

8 Edge Cases to Handle

1. **An empty batch.** What happens when a batch is `[]`? Your `Logic Trace` (Section 10) must explain, in your own words, why this can never cause your program to hang or error out.
2. **A batch of only control strings,** e.g., `["ERR", "ERR"]`. Confirm your `previous_value` tracking correctly stays at `None` throughout, with no delta comparisons attempted.
3. **`"STOP"` as the very first reading** in a batch. Confirm your shutdown logic still fires correctly even when no numeric readings have been processed yet in that batch.

9 Suggested Time Allocation

Phase	Description	Duration
Implementation	Core auditor, using the worked trace to self-check	55 minutes
Testing	Edge cases from Section 8, plus the Rolling Delta bonus	20 minutes
Documentation & Submission	Logic Trace comment block, final write-up	15 minutes
Total		**90 minutes**

10 Deliverables

1. **Source Code:** A single file named `anomaly_auditor.py`.
2. **Logic Trace:** A commented block at the top of your file explaining, in your own words, why your loop cannot get stuck if a batch is empty. (Hint: a Python `for` loop is bounded by the length of the sequence it iterates over — unlike a C-style loop with a manual index and condition, there is no way for it to spin indefinitely on zero elements.)
3. **Submission Documentation:** A short note (100–150 words), included either as an extended docstring or a separate paragraph, explaining how your `shutdown_triggered` flag ensures the outer loop's `else` clause fires only when no `"STOP"` signal was ever seen.

Academic Integrity

You may discuss approaches with peers during the supervised practical session, but the code and documentation you submit must be written independently and must reflect your own understanding. Submissions found to be substantially copied will be treated in accordance with the institute's academic integrity policy.